DEVELOPMENT AND PERFORMANCE ANALYSIS OF A DECISION TREE-BASED CLASSIFICATION MODEL FOR URBAN WATER QUALITY ASSESSMENT

Course Code & Name: ITA06 – Machine Learning Slot – A

Assignment Type: Machine Learning Project Report

Course Outcomes: CO1–CO5

Bloom's Taxonomy: L4 – Analyse; L5 – Evaluate; L6 – Create

SDG Mapping: SDG 6 – Clean Water and Sanitation; SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities; SDG 12 – Responsible Consumption and Production

Abstract

Urban water quality monitoring involves analysing several physical and chemical parameters to determine whether a water sample is suitable for its intended use. Manual assessment of large numbers of samples can be time-consuming. This project develops an interpretable Decision Tree classification model for urban water-quality assessment. Water-quality attributes such as pH, turbidity, dissolved oxygen, electrical conductivity, total dissolved solids and temperature are treated as input features, while the water-quality category is used as the target concept. Pandas is used for data inspection, filtering, sorting, grouping and preprocessing. A Decision Tree is constructed using a heuristic criterion such as Gini impurity or Information Gain. The learned tree is visualised and converted into representative IF–THEN rules. Model performance is evaluated using Accuracy, Precision, Recall, F1-score and a Confusion Matrix. The report also discusses hypothesis space, inductive bias, heuristic search, overfitting, underfitting and generalisation. The approach is suitable for practical monitoring because Decision Trees provide transparent rules that can support rapid screening and further investigation.

1. Introduction

Water quality is an important component of public health, environmental protection and sustainable urban development. Urban water management authorities collect samples from reservoirs, treatment facilities and distribution networks. Each sample may contain multiple measurements including pH, turbidity, dissolved oxygen, conductivity, total dissolved solids and temperature. A machine-learning classifier can learn relationships between these measurements and previously assigned quality classes, allowing new samples to be screened automatically.

2. Problem Definition

The task is formulated as a supervised multi-class classification problem. Given a water sample represented by a feature vector X = {pH, turbidity, dissolved oxygen, conductivity, TDS, temperature, ...}, the model predicts a target class Y representing the water-quality condition. The system should be accurate, interpretable and computationally practical.

3. Objectives

• Select and prepare a suitable publicly available water-quality dataset.
• Inspect, filter, sort and analyse the data using Pandas.
• Formulate water-quality assessment as a concept-learning problem.
• Identify the hypothesis space and inductive bias.
• Select important attributes using a Decision Tree heuristic.
• Construct and visualise the Decision Tree.
• Extract representative IF–THEN decision rules.
• Predict unseen water samples.
• Evaluate Accuracy, Precision, Recall, F1-score and Confusion Matrix.
• Analyse reliability, complexity, overfitting and generalisation.
• Relate the solution to relevant Sustainable Development Goals.

4. Dataset Description

A suitable publicly available water-quality dataset should contain sufficient observations and documented measurement attributes. Recommended attributes include pH, turbidity, dissolved oxygen, electrical conductivity, total dissolved solids, temperature, nitrate and chloride. The target variable is defined before training as a quality category such as Good, Moderate or Poor. If the selected public dataset provides a continuous water-quality index rather than classes, the index can be converted into clearly documented quality

categories using an appropriate domain-based threshold scheme. The exact thresholds and source must be reported in the final experimental submission.

5. Learning Problem and Concept Representation

The target concept is **Water Quality Class**. Each example consists of attribute-value pairs and a corresponding class label. A hypothesis is a rule or tree that maps input attributes to one of the target classes. The Decision Tree represents the learned concept through a hierarchy of tests. For example, a branch can test turbidity first, followed by pH or TDS, until a class is assigned.

6. Hypothesis Space and Inductive Bias

The hypothesis space consists of possible Decision Trees that can be formed using the available attributes and split points. It can be extremely large because different features, thresholds and tree structures can be combined. The learning algorithm searches this space heuristically rather than exhaustively. Inductive bias is introduced by preferences such as selecting the split that produces the greatest reduction in impurity, stopping when nodes are sufficiently pure, limiting tree depth and pruning unnecessary branches. These assumptions help the model generalise to unseen samples.

7. Heuristic Attribute Selection

Decision Trees require a criterion for selecting the next attribute. Two common choices are Information Gain and Gini impurity. Information Gain is based on entropy: $IG(S,A) = H(S) - \Sigma v\ (|Sv|/|S|)H(Sv)$. Entropy is $H(S) = -\Sigma i\ pi\ log2(pi)$. Gini impurity is $Gini(S) = 1 - \Sigma i\ pi^2$. The preferred attribute is the one producing the greatest reduction in impurity. In a practical implementation, the Gini criterion can be selected because it is directly supported by scikit-learn's DecisionTreeClassifier and is computationally efficient.

8. Decision Tree Construction

The training dataset is divided into training and testing subsets. The algorithm begins with all training samples at the root. For every candidate feature and threshold, impurity reduction is evaluated. The best split is selected, the data are partitioned, and the procedure is recursively repeated for each child node. Recursion stops when a node is sufficiently pure, when a depth limit is reached, when too few samples remain, or when no useful split is available. The majority class or pure class at a leaf becomes the prediction.

9. Pseudocode

INPUT: Water-quality dataset D
OUTPUT: Trained Decision Tree T and evaluation results

1. Load D using Pandas.
2. Inspect columns, data types, missing values and duplicates.
3. Clean invalid or missing records using documented preprocessing.
4. Separate features X and target y.
5. Encode categorical target values if required.
6. Split D into training and testing sets.
7. For each candidate split, calculate impurity reduction.
8. Select the best feature/threshold.
9. Partition the training data.
10. Recursively repeat steps 7–9 for child nodes.
11. Stop when stopping conditions are met.
12. Assign a class to each leaf.

13. Predict the test samples.
14. Calculate Accuracy, Precision, Recall and F1-score.
15. Generate the Confusion Matrix.
16. Visualise the tree and extract decision rules.
17. Analyse complexity, reliability and generalisation.

10. Data Preparation using Pandas

Pandas operations should include loading the CSV file, displaying the first and last records, checking dimensions and data types, detecting missing values, removing duplicates, filtering samples using multiple conditions, sorting values in ascending and descending order, grouping samples by quality class and calculating summary statistics. Numerical features should be checked for inconsistent values and impossible measurements. The preprocessing decisions must be documented so that the experiment is reproducible.

11. Python Implementation

The implementation can use Pandas for data handling, NumPy for numerical operations, scikit-learn for train/test splitting and Decision Tree learning, Matplotlib for plots and scikit-learn metrics for evaluation. The following compact code demonstrates the core model. File and target-column names should be changed to match the selected dataset.

12. Core Python Program

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay

df = pd.read_csv("water_quality.csv")

print(df.head())
print(df.info())
print(df.isnull().sum())

df = df.drop_duplicates()
df = df.dropna()

features = ["pH", "Turbidity", "Dissolved_Oxygen",
            "Conductivity", "TDS", "Temperature"]
X = df[features]
y = df["Water_Quality"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

model = DecisionTreeClassifier(
    criterion="gini", max_depth=5, random_state=42
)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

print("\nDecision Rules:")
print(export_text(model, feature_names=features))

plt.figure(figsize=(16, 9))
```

```
plot_tree(model, feature_names=features,
          class_names=model.classes_, filled=True, rounded=True)
plt.title("Decision Tree for Urban Water Quality")
plt.show()

ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.title("Confusion Matrix")
plt.show()

new_sample = [[7.2, 1.5, 7.0, 450, 250, 25]]
print("Predicted class:", model.predict(new_sample)[0])
```

13. Menu-Driven Functionality

A menu-driven program improves usability and directly addresses the implementation rubric. Suggested options are: (1) Load Dataset, (2) Display Dataset, (3) Dataset Information, (4) Handle Missing Values, (5) Filter Samples, (6) Sort Data, (7) Group and Analyse, (8) Train Decision Tree, (9) Feature Importance, (10) Visualise Tree, (11) Predict New Sample, (12) Confusion Matrix, (13) Model Evaluation, (14) Display Decision Rules, and (15) Exit. Input validation should prevent crashes when a user enters an invalid menu option or incomplete sample values.

14. Decision Tree Visualisation and Analysis

The tree visualisation should show the selected feature, split threshold, impurity, number of samples and predicted class at each node. The root feature is generally the most influential first split under the selected criterion. A shallow, interpretable tree is preferred for operational use because water-quality personnel can inspect and understand its decisions. Excessive depth should be avoided because it can memorise noise in the training data.

15. Representative Decision Rules

Representative rules can be extracted from the trained tree. Examples of the format are:

Rule 1: IF Turbidity is low AND pH is within the acceptable range THEN Water Quality = Good.
Rule 2: IF Turbidity is high AND TDS is high THEN Water Quality = Poor.
Rule 3: IF Dissolved Oxygen is low AND conductivity is high THEN Water Quality = Poor or Moderate.

These are illustrative rule forms; the final report must use the actual rules generated from the selected dataset and trained tree.

16. Performance Evaluation

The model should be evaluated on unseen test samples. Accuracy measures the overall fraction of correct predictions. Precision measures the proportion of predicted samples of a class that are actually in that class. Recall measures how many samples belonging to a class are correctly detected. F1-score is the harmonic mean of precision and recall. A Confusion Matrix shows correct and incorrect predictions by class. For water-quality screening, recall for the Poor class can be particularly important because failing to identify a poor-quality sample may have greater practical consequences than a false alarm.

17. Reliability, Overfitting and Generalisation

A Decision Tree with excessive depth may overfit the training data, resulting in high training accuracy but lower test performance. Underfitting can occur when the tree is too shallow to represent meaningful relationships. Reliability should therefore be examined by comparing training and testing performance and, where possible, using cross-validation. Parameters such as max_depth, min_samples_split and

min_samples_leaf can control complexity. A stable test score and a reasonable gap between training and test performance indicate better generalisation.

18. Feature Importance

Decision Tree feature importance values can be inspected using model.feature_importances_. Larger values indicate that the feature contributed more to impurity reduction across the tree. The result should be interpreted carefully: feature importance indicates predictive contribution within the trained model and does not by itself prove causation. Correlated water-quality parameters can also distribute importance across several features.

19. Neural Network Concept Analysis (CO3)

Perceptrons are simple linear classifiers. A Multilayer Perceptron extends this idea through hidden layers and nonlinear activation functions. Backpropagation computes gradients and updates network weights to reduce prediction error. Neural networks can model complex nonlinear relationships but generally require more tuning and provide less transparent decision rules than a small Decision Tree. For an urban water-quality screening system where interpretability is a key requirement, the Decision Tree offers an important practical advantage.

20. Genetic Algorithm and Hypothesis-Space Search (CO4)

A Genetic Algorithm can search a hypothesis space using a population of candidate solutions. Each candidate is evaluated using a fitness function, and selection, crossover and mutation create new candidates. For this project, a GA could optimise Decision Tree hyperparameters or feature subsets. However, GA-based optimisation generally requires additional computational evaluations. A heuristic Decision Tree learner is simpler and more directly interpretable for the stated application.

21. Computational Feasibility

Decision Trees are computationally practical for tabular water-quality data. Training is generally fast for moderate-sized datasets, prediction for individual samples is very fast, and the learned structure can be stored and deployed easily. Memory and computation increase with the number of samples, features and possible split points. Limiting tree depth and removing irrelevant features can help maintain a compact model.

22. SDG Relevance

The project directly supports SDG 6 by contributing to water-quality monitoring and clean-water management. It supports SDG 9 through the use of data-driven technological infrastructure, SDG 11 through intelligent urban service management, and SDG 12 through improved monitoring and responsible management of water resources. The classifier can serve as a screening tool that helps prioritise samples for laboratory confirmation or further treatment.

23. Limitations

The model depends on the quality and representativeness of historical measurements. Sensor errors, missing values, seasonal variation, geographic differences and changes in water-treatment processes can affect generalisation. A Decision Tree can also be sensitive to small changes in training data. Therefore, the model should support—not replace—laboratory testing, regulatory procedures and expert judgement. Thresholds used to define Good, Moderate and Poor classes must be documented and justified.

24. Final Evaluation and Recommendation

The Decision Tree is recommended as a practical baseline for urban water-quality classification because it combines predictive modelling with high interpretability. Its IF–THEN rules can be communicated to

water-management personnel, while standard classification metrics provide quantitative evidence of performance. The final model should be selected only after comparing training and test performance, examining the confusion matrix, checking class balance and confirming that the tree is not excessively complex. Where misclassification of poor-quality samples is especially costly, recall for the Poor class should receive particular attention.

25. Reflection and Learning Outcomes

This project demonstrates how a real-world monitoring problem can be formulated as a concept-learning task. The work connects hypothesis-space search and inductive bias with a practical Decision Tree implementation. Pandas operations provide the foundation for reliable dataset preparation, while heuristic splitting makes learning computationally feasible. The project also highlights the trade-off between predictive performance and interpretability. Comparing Decision Trees with Neural Networks and Genetic Algorithms provides a broader understanding of machine-learning strategies and their suitability for different engineering applications.

26. Conclusion

A Decision Tree-based classification system provides an interpretable approach for urban water-quality assessment. By combining systematic data preparation, heuristic attribute selection, recursive tree construction, decision-rule extraction and quantitative evaluation, the proposed solution can classify unseen samples and assist authorities in prioritising investigation. The final experimental results should be reported using the actual selected dataset, including Accuracy, Precision, Recall, F1-score, Confusion Matrix, tree depth and number of leaves. With appropriate validation and periodic retraining, the model can serve as a useful decision-support component in intelligent urban water management.

27. GitHub Upload Structure

Recommended repository structure:

urban-water-quality-decision-tree/
■■■ README.md
■■■ data/
■ ■■■ water_quality.csv
■■■ src/
■ ■■■ water_quality_decision_tree.py
■■■ results/
■ ■■■ decision_tree.png
■ ■■■ confusion_matrix.png
■ ■■■ feature_importance.png
■■■ report/
■ ■■■ project_report.pdf
■■■ requirements.txt

The README should describe the dataset source, preprocessing, installation requirements, execution steps, model criterion, evaluation results and limitations.

28. Deliverables Checklist

■ Pseudocode
■ Dataset preparation and Pandas analysis
■ Decision Tree implementation
■ Heuristic attribute-selection analysis

■ Tree visualisation
■ Decision rules
■ Unseen-sample prediction
■ Accuracy, Precision, Recall and F1-score
■ Confusion Matrix
■ Reliability, overfitting and generalisation analysis
■ Menu-driven functionality
■ SDG mapping and reflection
■ GitHub repository upload

References

1. T. M. Mitchell, *Machine Learning*, McGraw-Hill, 1997.
2. J. R. Quinlan, *C4.5: Programs for Machine Learning*, Morgan Kaufmann, 1993.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
5. W. McKinney, *Python for Data Analysis*, O’Reilly Media.
6. United Nations, Sustainable Development Goals, especially SDG 6, SDG 9, SDG 11 and SDG 12.